1st & 2nd lecture (12/02/2023).

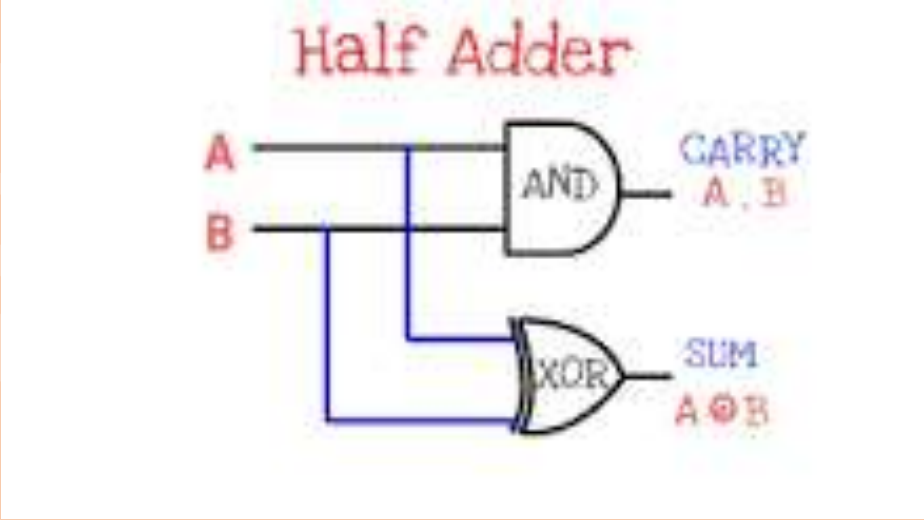
Topics:

- **Combinational logic:** combinational logic is one in which memory allocation is not required (1+1=2).
- **Logic, systems, digital system, Analog system, digital logic, analog logic.**
  - ✓ **Logic:** It is defined as a declarative sentence that is either True or False.
  - ✓ **Systems:** A computer system consists of hardware components that have been carefully chosen so that they work well together.
  - ✓ **Digital system:** Digital systems are designed to store, process, and communicate information in digital form.
  - ✓ **Analog system:** A system that uses continuous time signal for processing is known as an analog system.
  - ✓ **Digital logic:** Digital Circuits operate on discretely variable signals or Digital signals.
  - ✓ **Analog logic:** Analog circuits operate on continuously variable signals also known as Analog Signals.
- **Computer science:** It is the combination of applied math and applied physics.
- **Digital logic.**
  1. **Combinational logic:** combinational logic is one in which memory allocation is not required (1+1=2).
  2. **Sequential logic:** sequential logic is one in which memory allocation is required (17+2=19).
- **Design procedure.**
  1. Identify the problem.
  2. Determine the input and output variable.
  3. Input and output variable assigned with letter symbols.
  4. Create truth table.
  5. Derive logic from truth table.
  6. Design.
- **Adder:** An adder is a device that will add together two bits and give the result as the output.

1. **Half-adder:** Half Adder is a combinational logic circuit which is designed by connecting one EX-OR gate and one AND gate.
   - ✓ **Truth table:**

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

   - ➢ The sum (S) of the half-adder is the XOR of A and B. Thus,
     Sum, $S=A \oplus B=AB'+A'B$
   - ➢ The carry (C) of the half-adder is the AND of A and B. Therefore,
     Carry, $C=A \cdot B$
   - ✓ **Logic diagram:**



2. **Full-adder:** A full adder adds three one-bit binary numbers, two operands and a carry bit.

✓ **Truth table for full-adder:**

Inputs			Outputs	
A	B	C	S (Sum)	C (Carry)
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

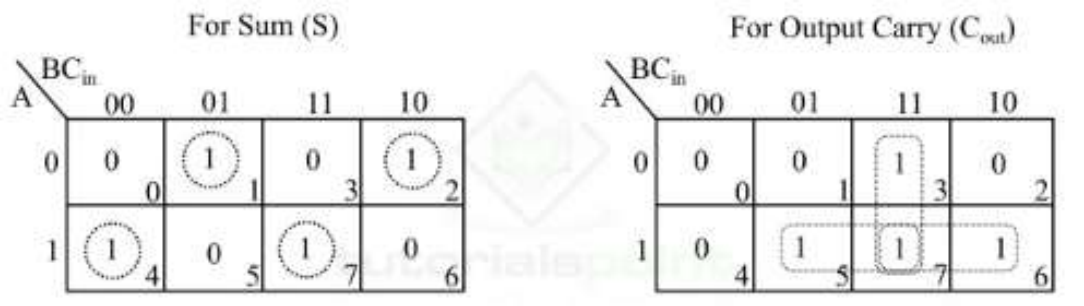
The sum (S) of the full-adder is the XOR of A, B, and C. Therefore,

$$\text{Sum, } S=A\oplus B\oplus C=A'B'C+A'BC'+AB'C'+ABC$$

The carry (C) of the half-adder is the AND of A and B. Therefore,

$$\text{Carry, } C=A'BC+AB'C+ABC'+ABC$$

✓ **Simplify using k-map:**



➢ **For sum there is no change in output:**

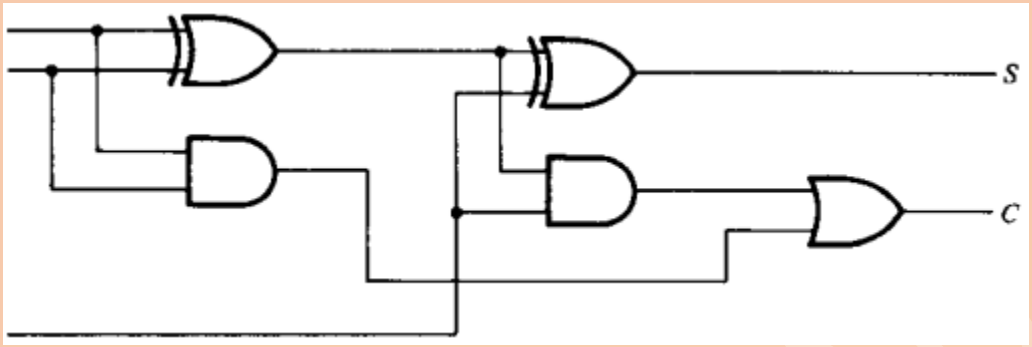
Sum, S =A'B'C+A'BC'+AB'C'+ABC

=A⊕B⊕C.

➢ **For carry:**

Carry, C=AB+BC+AC=AB+C(A+B)

✓ **Logic diagram:**



Sum, S =A'B'C+A'BC'+AB'C'+ABC

=A⊕B⊕C.

Carry, C=A'BC+AB'C+ABC'+ABC

=C(A'B+AB')+AB(C+C')

=C(A⊕B)+AB

3rd, 4th & 5th lecture(13/02/2023)

❖ **Subtractors:** Subtractor Circuit is a combinational logic circuit that performs subtraction on binary numbers.

1. **Half-subtractor:** The half-subtractor is a combinational circuit which is used to perform subtraction of two bits.
   - ✓ The truth table for the input-output relationships of a half-subtractor can now derived as follows:

x	y	B	D
0	0	0	0
0	1	1	1
1	0	0	1
1	1	0	0

   - ✓ **The Boolean function derived from the truth table:**

     D=x'y+xy'=x⊕y

     B=x'y
   - ✓ **Draw the circuit for D and B as you know.**

- **For better understanding click the link.If link is not clickable then copy the URL and paste it on your search engine search bar and hit enter button:**
  - ✓ https://youtu.be/SV4VTYWxKV4

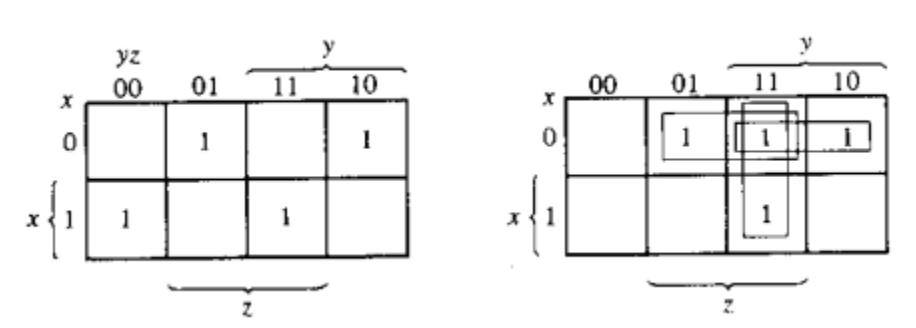
2. **Full-subtractor:** A full subtractor is a combinational circuit that performs subtraction involving three bits, namely A (minuend), B (subtrahend), and Bin (borrow-in).
   - ✓ **Truth table for full-subtractor:**

x	y	z	B	D
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	1	0
1	0	0	0	1
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

   - ✓ **Boolean function derived from the above truth table:**

     D=x'y'z+x'yz'+xy'z'+xyz

     B= x'y'z+x'yz'+x'yz+xyz
   - ✓ **Optimization using k-map:**



     After simplifying,

     D=unchangeable= x'y'z+x'yz'+xy'z'+xyz

     =x'(y'z+yz')+x(y'z'+yz)

     =x'(y⊕z)+x(y⊕z)'

     =x ⊕(y⊕z)

     B= x'y+x'z+yz
   - ✓ **Draw circuit diagram for D and B as you know.**

- **For better understanding click the link.If link is not clickable then copy the URL and paste it on your search engine search bar and hit enter button:**
  - ✓ https://youtu.be/dBXGGWbtt6U

❖ **Code conversion: BCD to the excess-3 code.(BCD to excess-n like 4)*********

✓ **Truth table for code-conversion:**

Input BCD				Output Excess-3 Code			
A	B	C	D	w	x	y	z
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0

✓ **Boolean function from above table:**

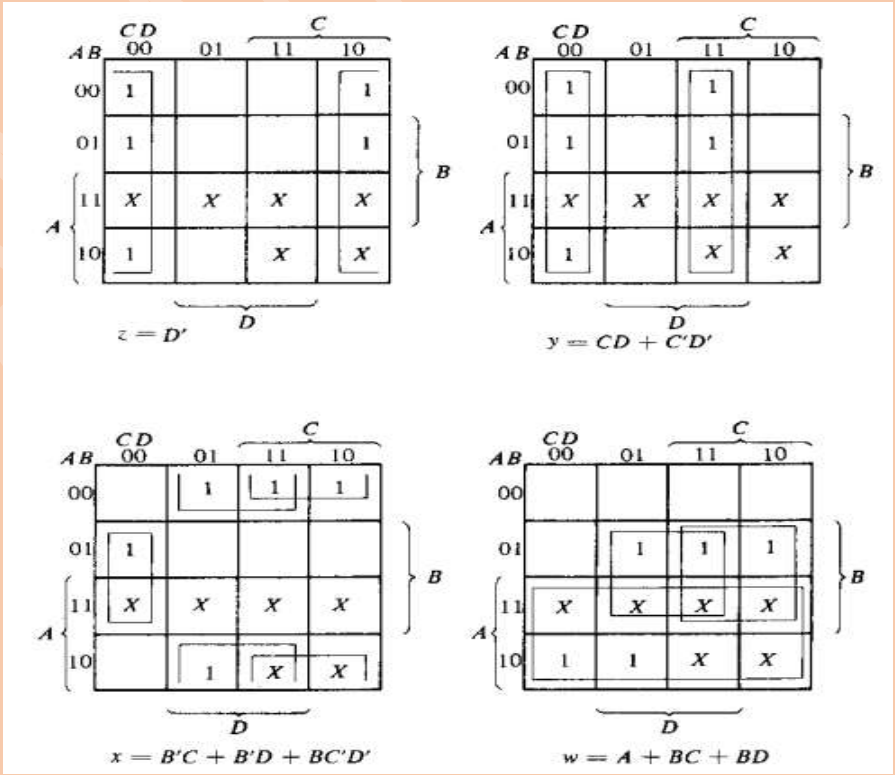
W=A'BC'D+A'BCD'+A'BCD+AB'C'D'+AB'C'D

X=A'B'C'D+A'B'CD'+A'B'CD+A'BC'D'+AB'C'D

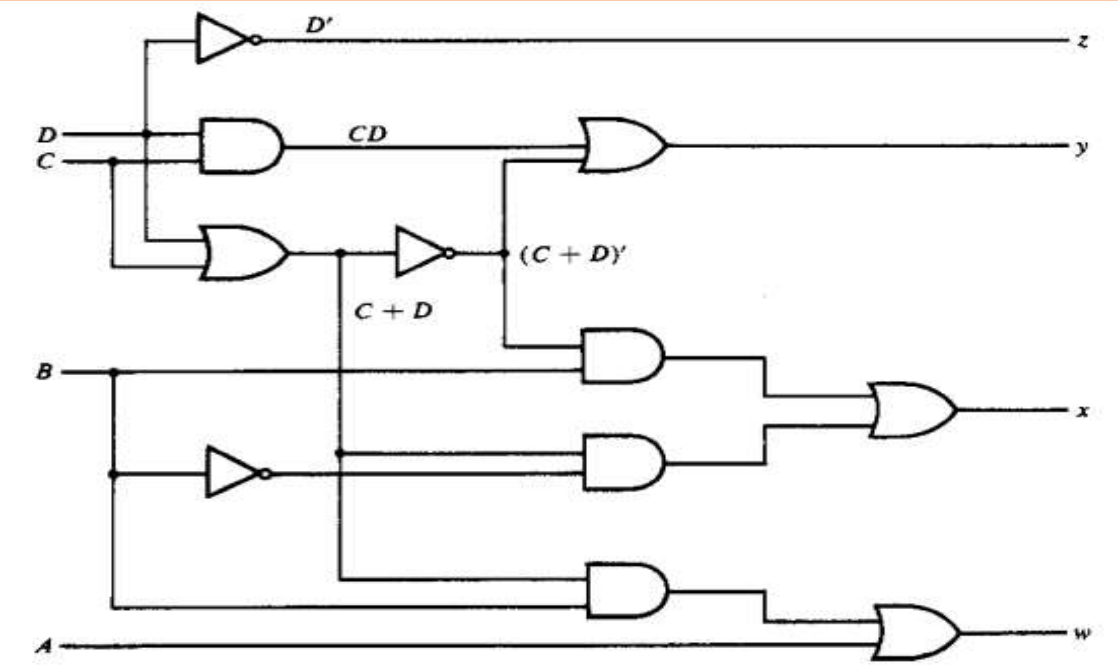
Y=A'B'C'D'+A'B'CD+A'BC'D'+A'BCD+AB'C'D'

Z=A'B'C'D'+A'B'CD'+A'BC'D'+A'BCD'+AB'C'D'

✓ **Simplify using k-map:**



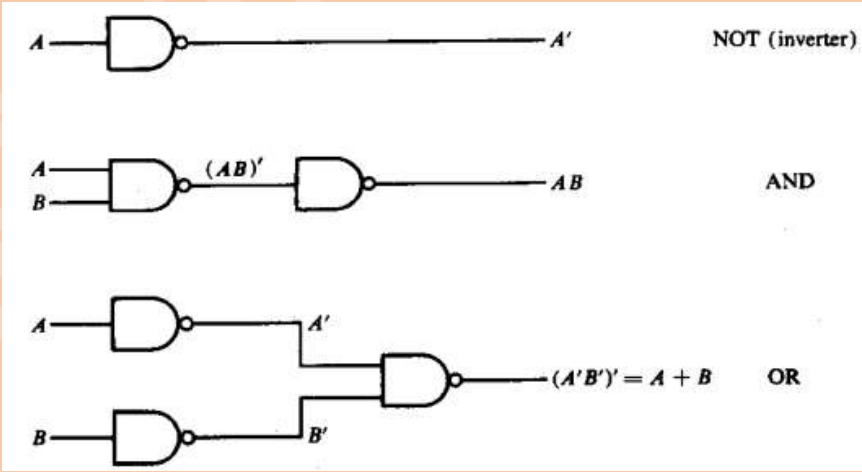
✓ **Logic diagram for above simplification:**



- **For better understanding click the link.If link is not clickable then copy the URL and paste it on your search engine search bar and hit enter button:**
  - ✓ https://youtu.be/LHw8TVk9iOY

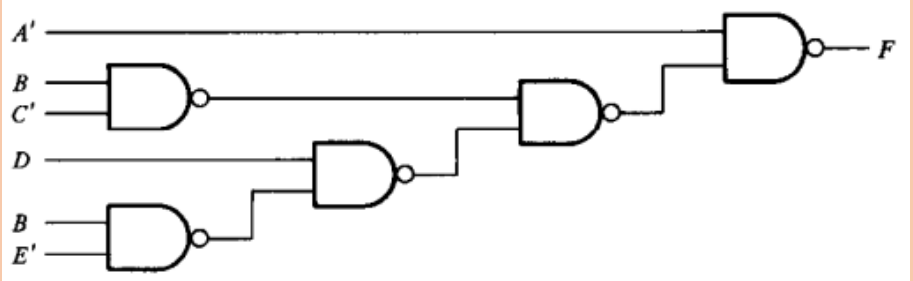
❖ **Multilevel NAND circuit.**

✓ **Universal gate:** Make AND, OR, NOT gate using NAND gate.

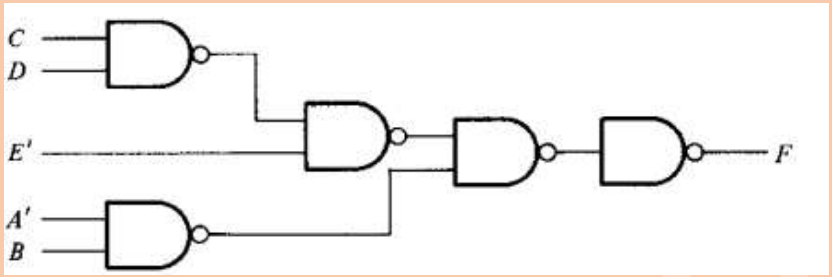


✓ **Boolean-function implementation using NAND gate:**

➢ **F=A+(B'+C)(D'+BE').**

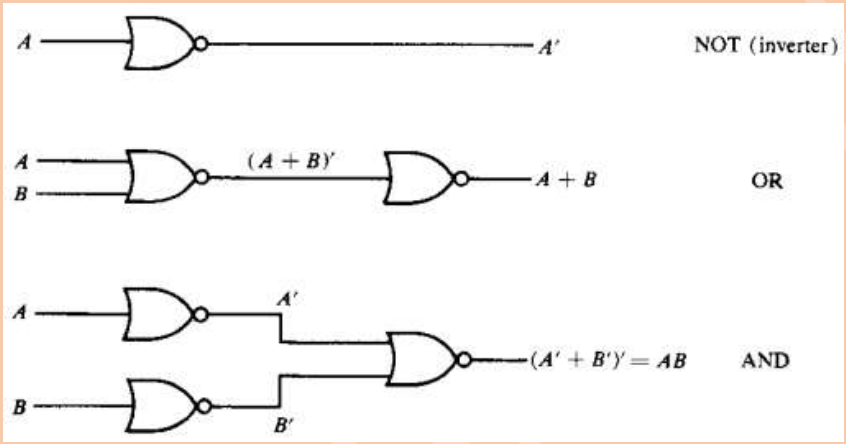


- **F=(CD+E)(A+B').**

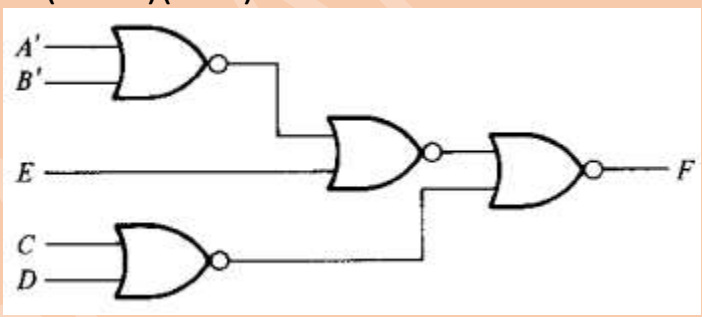


❖ Multilevel NOR Circuit.

- ✓ **Universal gate:** Make AND, OR, NOT gate using NOR gate.

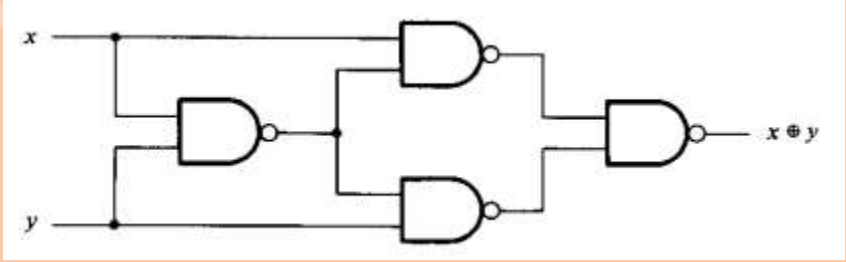


- ✓ **Boolean function implementation:**
  - F=(AB+E)(C+D)



❖ Exclusive or(Ex-OR) function:

- **Make a logic gate for the function X⊕Y only using NAND gate:*****



- **Odd function:**

$$A \oplus B \oplus C = (AB' + A'B)C' + (AB + A'B')C$$
$$= AB'C' + A'BC' + ABC + A'B'C$$
$$= \Sigma(1, 2, 4, 7)$$

- **For four variable Ex-OR operation:**

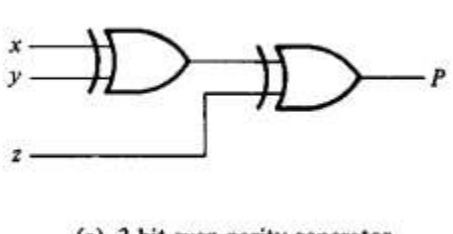
$$A \oplus B \oplus C \oplus D = (AB' + A'B) \oplus (CD' + C'D)$$
$$= (AB' + A'B)(CD + C'D') + (AB + A'B')(CD' + C'D)$$
$$= \Sigma\,(1, 2, 4, 7, 8, 11, 13, 14)$$

❖ Parity generator:

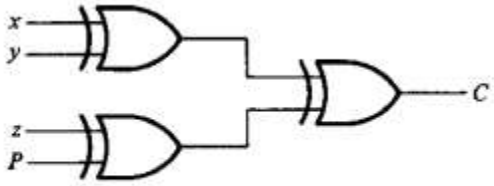
- **Even parity generator truth table:**

Three-Bit Message			Parity Bit
x	y	z	P
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

- **Logic diagram of parity generator and checker:**



(a) 3-bit even parity generator



(b) 4-bit even parity checker

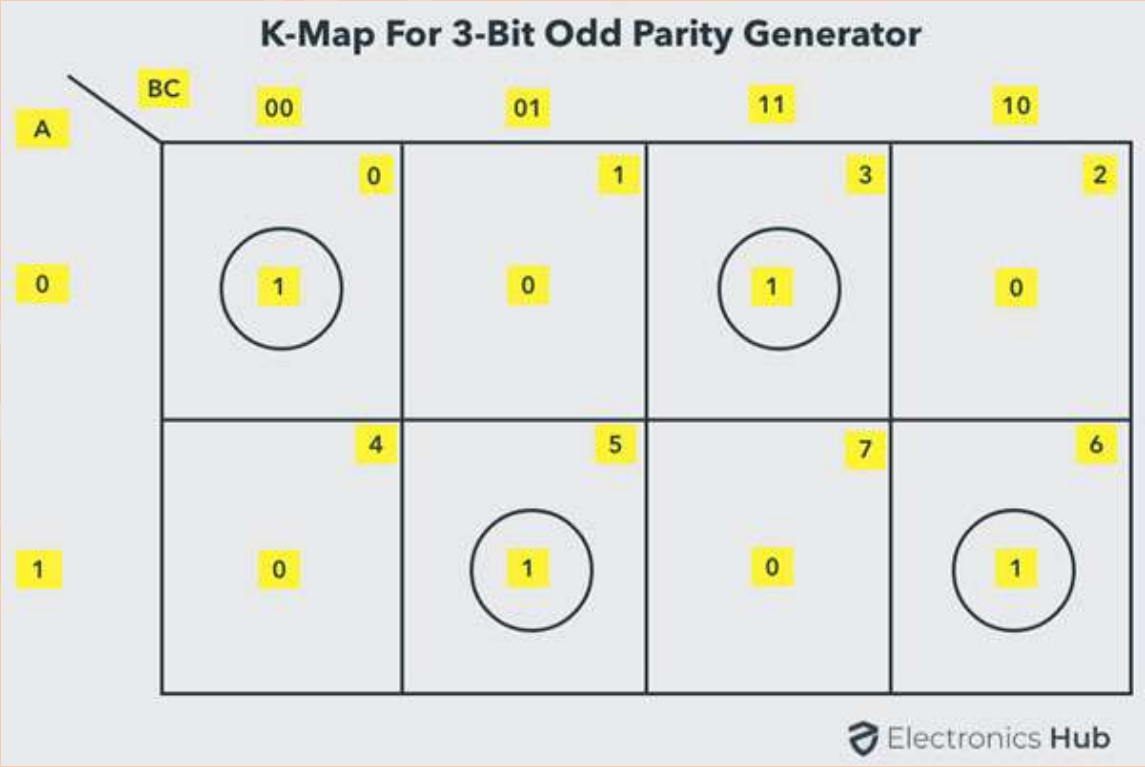
Lecture 6th & 7th. (16/02/2023)

❖ **Odd parity generator for 3-bit.**

✓ **Truth table:**

3-bit message			Odd parity bit generator (P)
A	B	C	Y
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

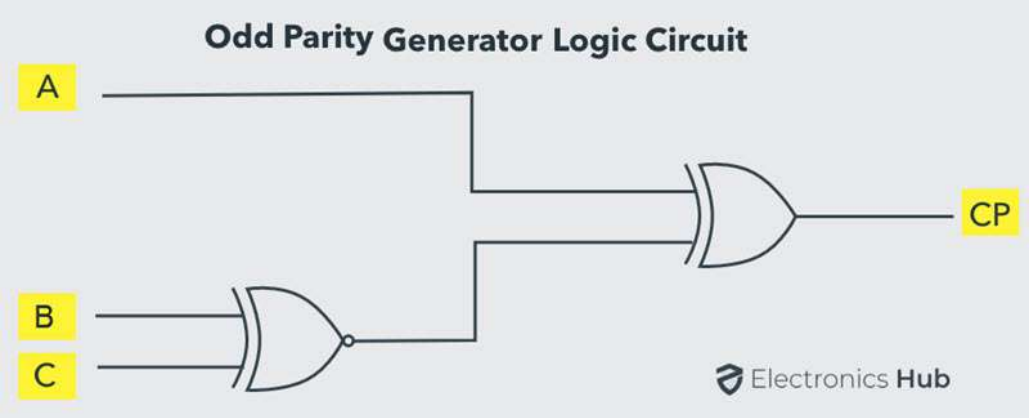
✓ **The truth table of the odd parity generator can be simplified by using K-map as:**



The output parity bit expression for this generator circuit is obtained as

$$P = A \oplus (B \oplus C)$$

✓ **Logic diagram:**

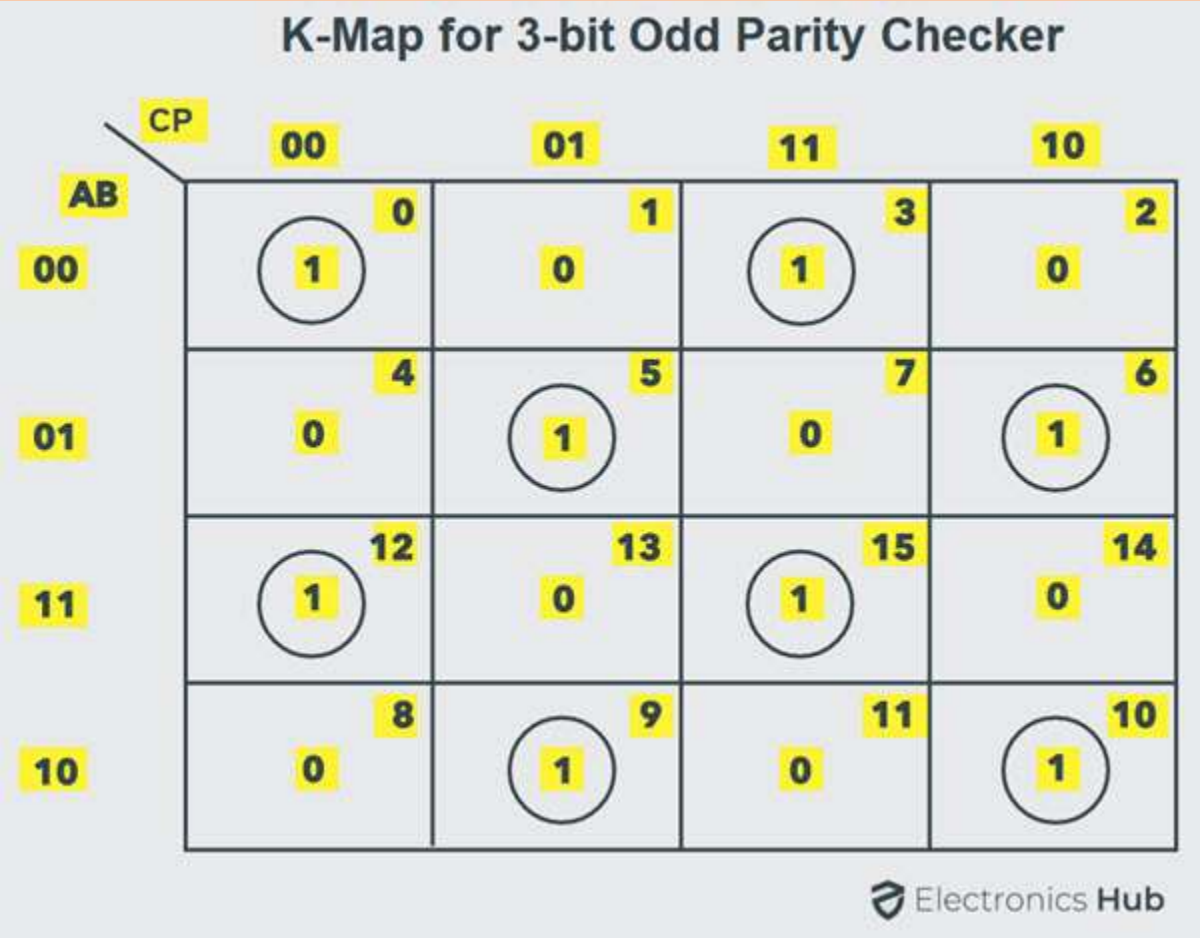


❖ **Odd parity generator for 4-bit:**

✓ **Truth table:**

4-bit received message				Parity error check C_p
A	B	C	P	
0	0	0	0	1
0	0	0	1	0
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	0
1	0	0	0	0
1	0	0	1	1
1	0	1	0	1
1	0	1	1	0
1	1	0	0	1
1	1	0	1	0
1	1	1	0	0
1	1	1	1	1

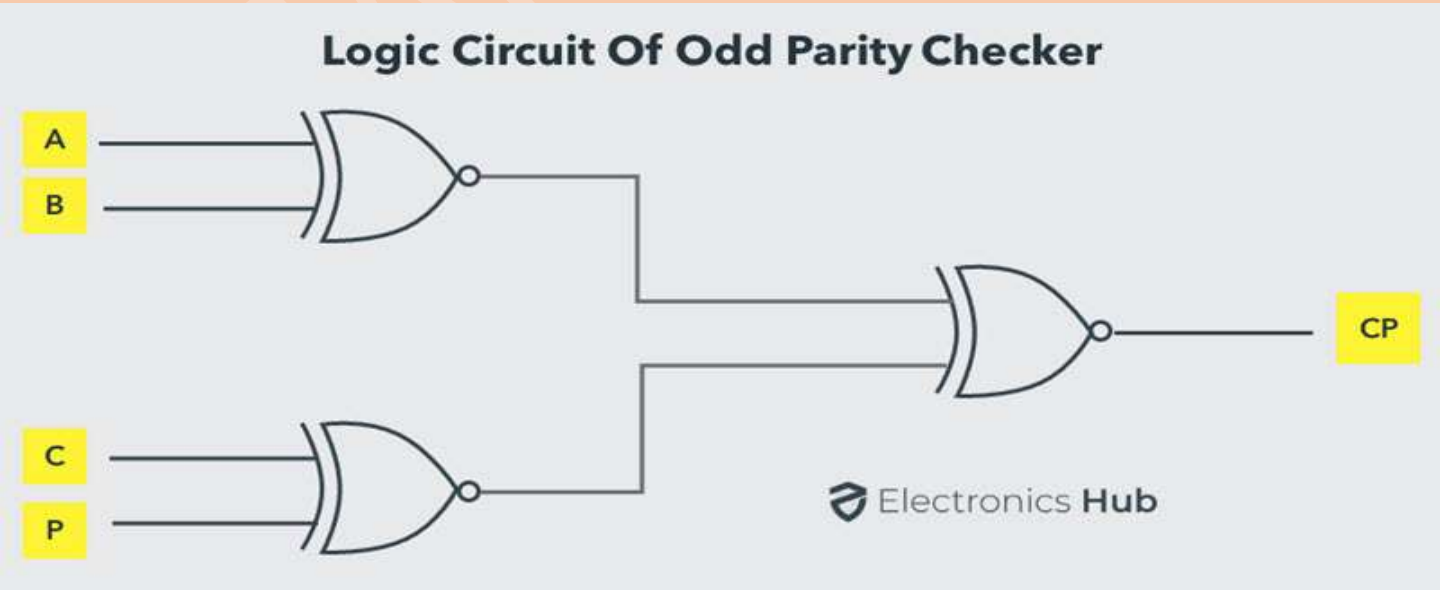
✓ **The expression for the PEC in the above truth table can be simplified by K-map as shown below:**



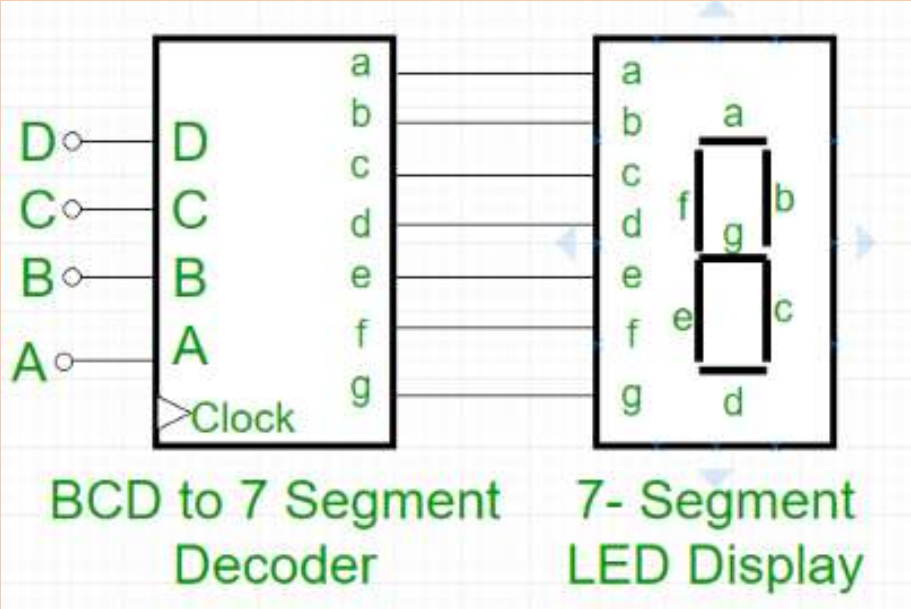
After simplification, the final expression for the PEC is obtained as

PEC ={ (A ⊕ B)' ⊕ (C ⊕ P)'}'

✓ **Logic diagram:**



❖ **Seven-segment decoder:**

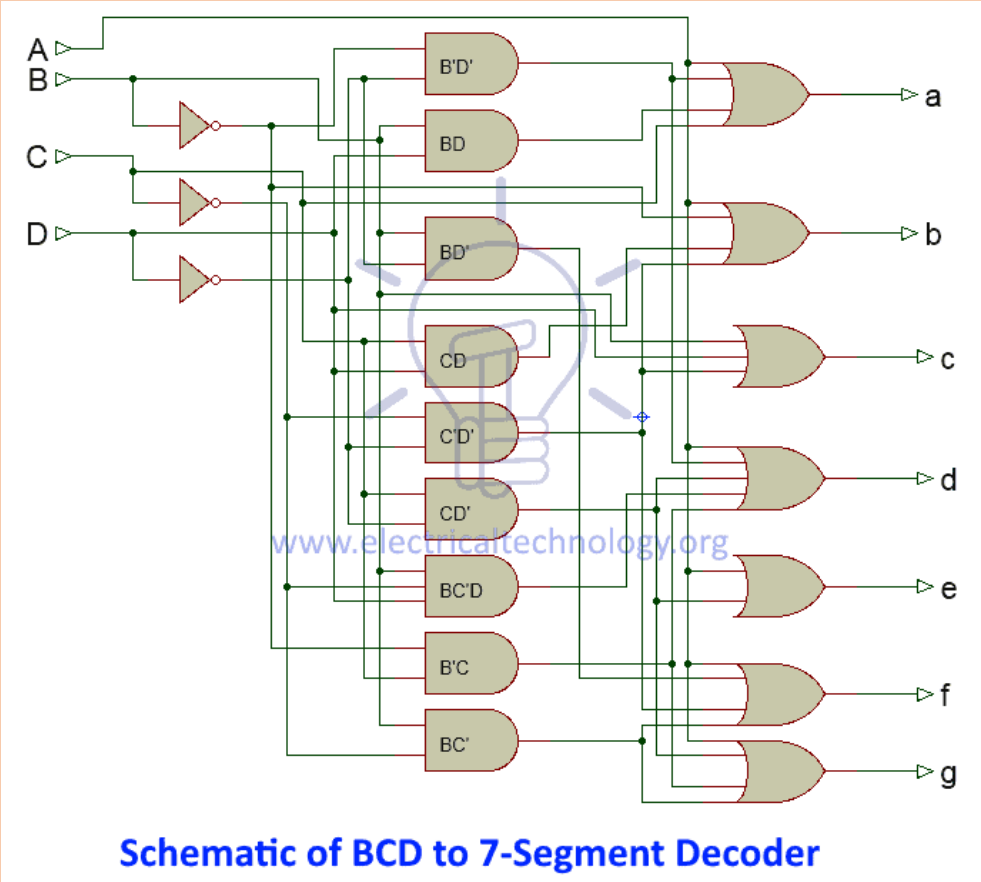


- **Truth table:**

	INPUT				OUTPUT						
Digits	A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	0	1	1	1	1	1	1	0
1	0	0	0	1	0	1	1	0	0	0	0
2	0	0	1	0	1	1	0	1	1	0	1
3	0	0	1	1	1	1	1	1	0	0	1
4	0	1	0	0	0	1	1	0	0	1	1
5	0	1	0	1	1	0	1	1	0	1	1
6	0	1	1	0	1	0	1	1	1	1	1
7	0	1	1	1	1	1	1	0	0	0	0
8	1	0	0	0	1	1	1	1	1	1	1
9	1	0	0	1	1	1	1	1	0	1	1

- **Simplify using k-map and we find:**
  - ✓ For a=A+C+BD+B’D’
  - ✓ For b=B'+C'D'+CD
  - ✓ For c=B+C'+D
  - ✓ For d=A+B'D'+B'C+CD'+BC'D
  - ✓ For e=B'D'+CD'
  - ✓ For f=A+BC'+BD'+C'D'
  - ✓ For g=A+BC'+B'C+CD'

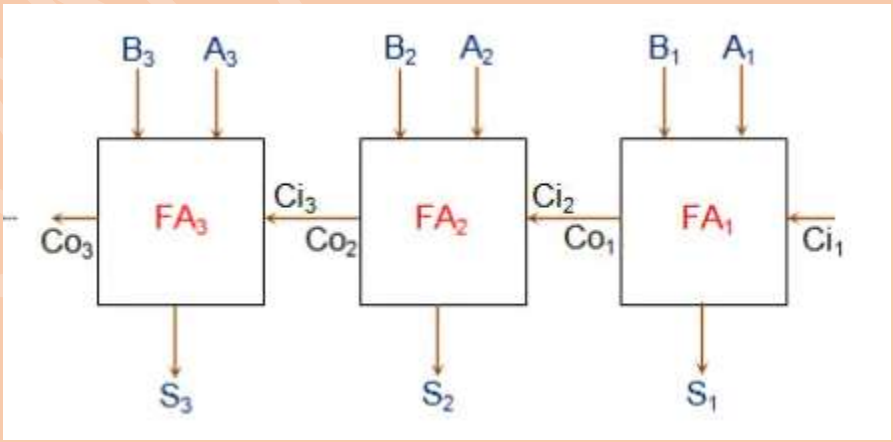
- **Logic diagram:**



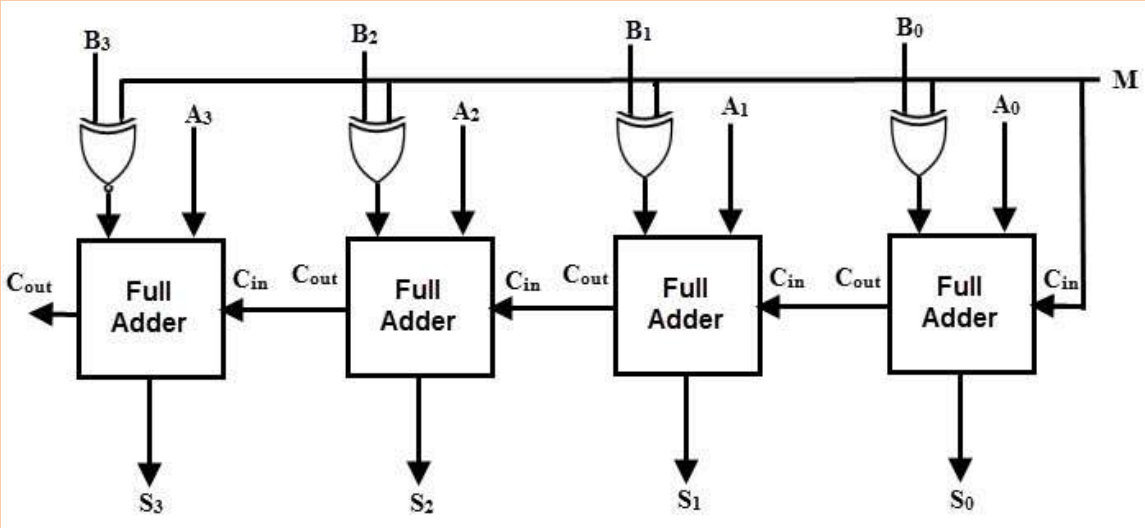
Schematic of BCD to 7-Segment Decoder

❖ **Binary adder:**

✓ **3-bit parallel adder:**



✓ **4-bit parallel adder:**



❖ **Carry propagation:** consider the full adder circuit we find.

$$P_i = A_i \oplus B_i$$

$$G_i = A_i B_i$$

the output sum and carry can be expressed as

$$S_i = P_i \oplus C_i$$

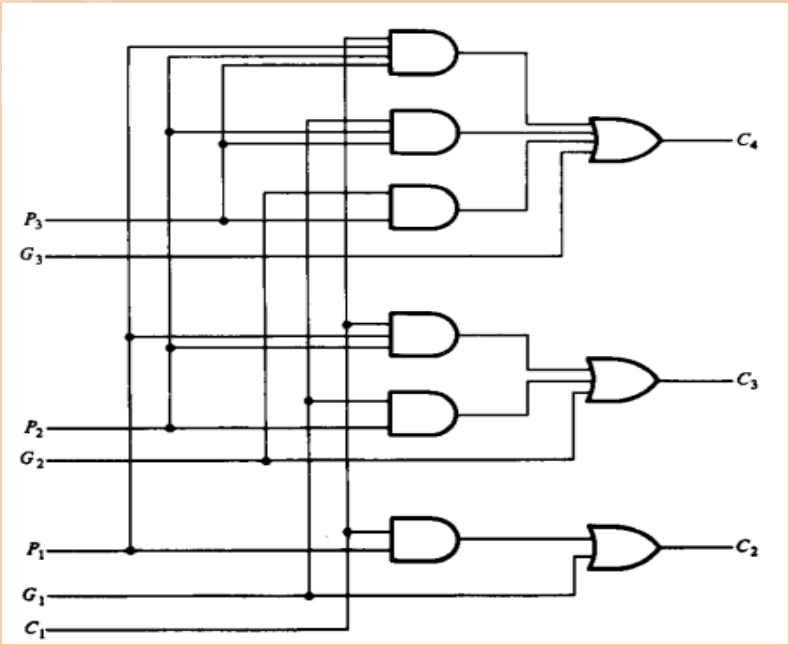
$$C_{i+1} = G_i + P_i C_i$$

$$C_2 = G_1 + P_1 C_1$$

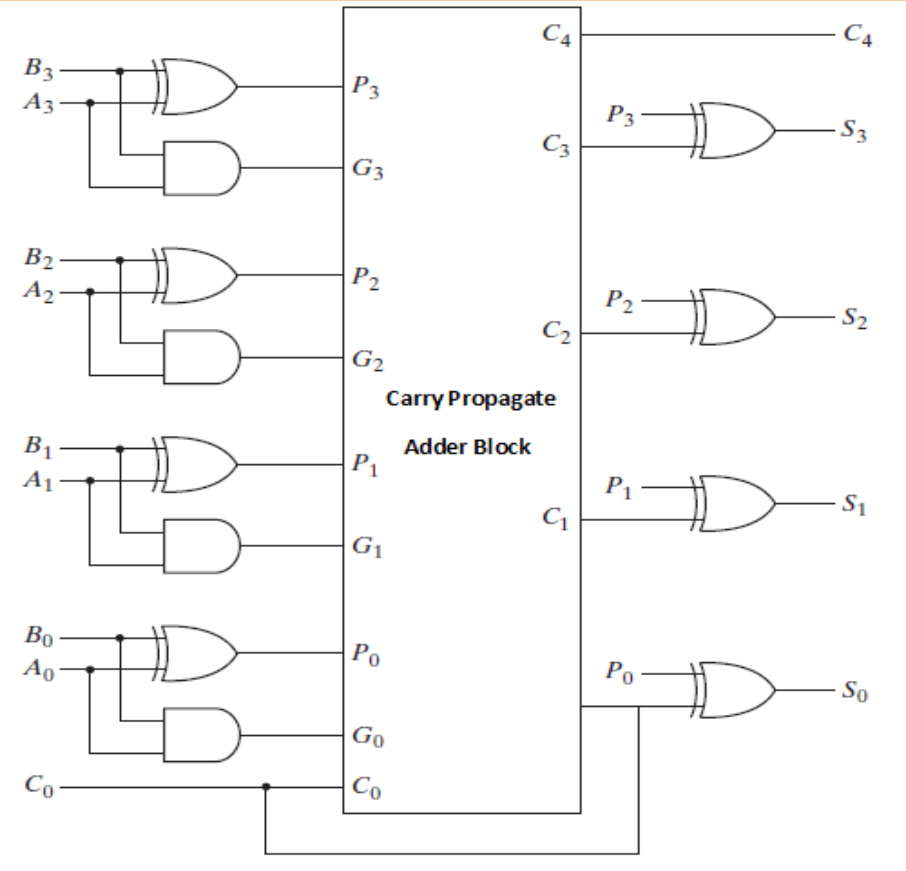
$$C_3 = G_2 + P_2 C_2 = G_2 + P_2(G_1 + P_1 C_1) = G_2 + P_2 G_1 + P_2 P_1 C_1$$

$$C_4 = G_3 + P_3 C_3 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 C_1$$

✓ **Logic diagram for above equation.**



✓ **4-bit full-adder with carry:**



8th, 9th & 10th Lecture.(20/02/2023)

❖ **BCD adder.**

✓ **Truth table:**

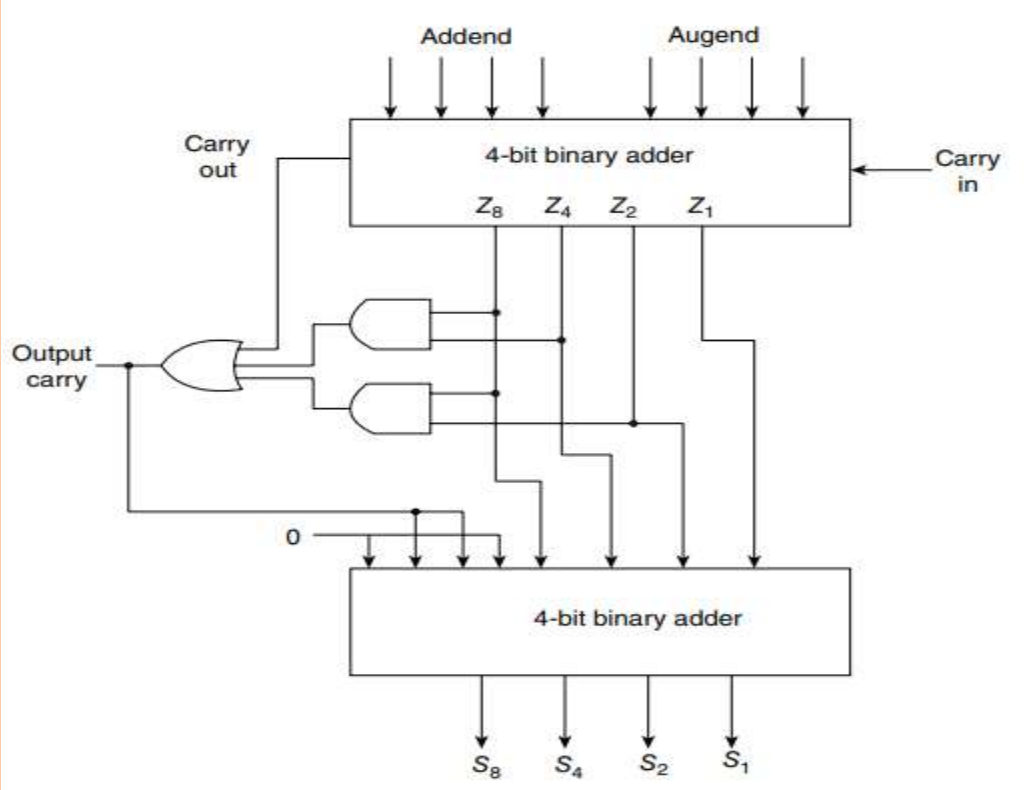
Binary sum					BCD sum					Decimal
K	Z_8	Z_4	Z_2	Z_1	C	S_8	S_4	S_2	S_1	
0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	2
0	0	0	1	1	0	0	0	1	1	3
0	0	1	0	0	0	0	1	0	0	4
0	0	1	0	1	0	0	1	0	1	5
0	0	1	1	0	0	0	1	1	0	6
0	0	1	1	1	0	0	1	1	1	7
0	1	0	0	0	0	1	0	0	0	8
0	1	0	0	1	0	1	0	0	1	9
0	1	0	1	0	1	0	0	0	0	10
0	1	0	1	1	1	0	0	0	1	11
0	1	1	0	0	1	0	0	1	0	12
0	1	1	0	1	1	0	0	1	1	13
0	1	1	1	0	1	0	1	0	0	14
0	1	1	1	1	1	0	1	0	1	15
1	0	0	0	0	1	0	1	1	0	16
1	0	0	0	1	1	0	1	1	1	17
1	0	0	1	0	1	1	0	0	0	18
1	0	0	1	1	1	1	0	0	1	19

The condition for a correction and an output carry can be expressed by the Boolean function: $C = K + Z_8Z_4 + Z_8Z_2$

- ✓ **Draw circuit diagram as we know.**

❖ **Magnitude comparator:** A magnitude digital comparator is a combinational circuit that compares two digital or binary numbers in order to find out whether one binary number is equal, less than or greater than the other binary number.

- ✓ **Block diagram of a BCD adder:**



Consider two numbers, A and B, with four digits each. Write the coefficients of the numbers with descending significance as follows: A = A3 A2 A1 A0, B = B3 B2 B1 B0 where each subscripted letter represents one of the digits in the number. The two numbers are equal if all pairs of significant digits are equal **i.e, if $A_3 = B_3$ and $A_2 = B_2$ and $A_1 = B_1$ and $A_0 = B_0$**. When the numbers are binary, the digits are either 1 or 0 and the equality relation of each pair of bits can be expressed logically with an equivalence function:

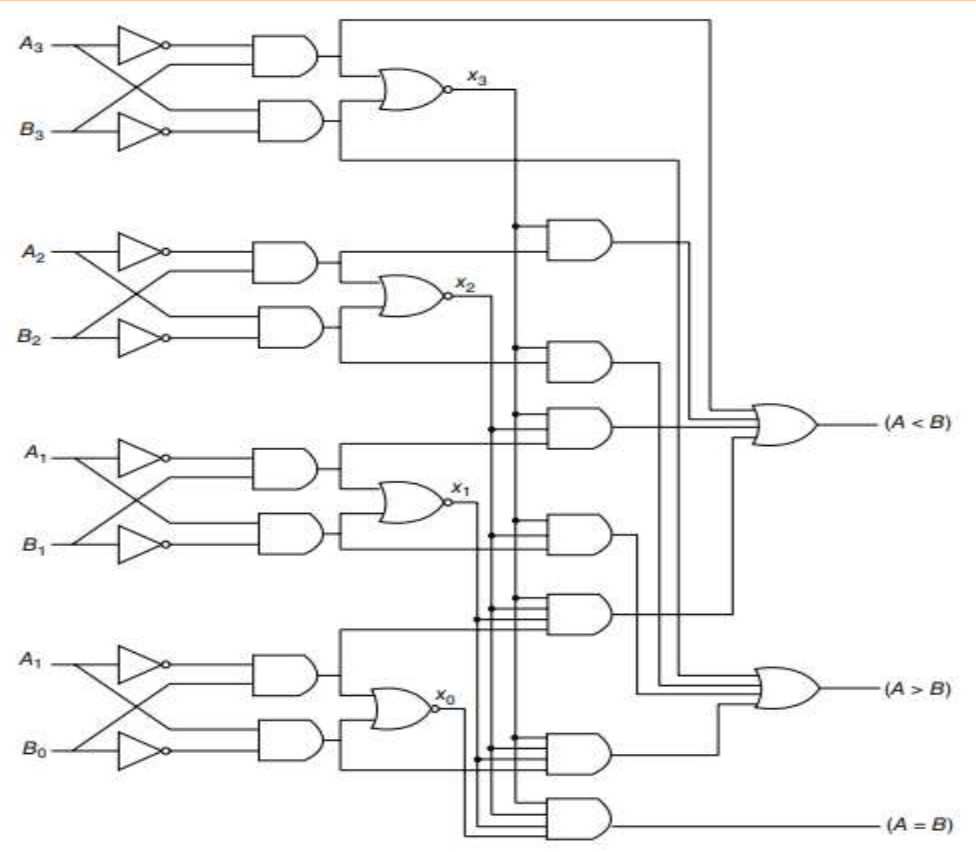
- **$x_i = A_iB_i + A'_iB'_I$; i = 0, 1, 2, 3**

The sequential comparison can be expressed logically by the following two Boolean functions:

- ✓ **$(A > B) = A_3B'_3 + x_3A_2B'_2 + x_3x_2A_1B'_1 + x_3x_2x_1A_0B'_0$**
- ✓ **$(A < B) = A'_3B_3 + x_3A'_2B_2 + x3x2A'_1B1 + x_3x_2x_1A'_0B_0$**

the symbols (A > B) and (A < B) are binary output variables which are equal to 1 when A > B or A < B, respectively.

- **4-bit magnitude comparator circuit diagram:**

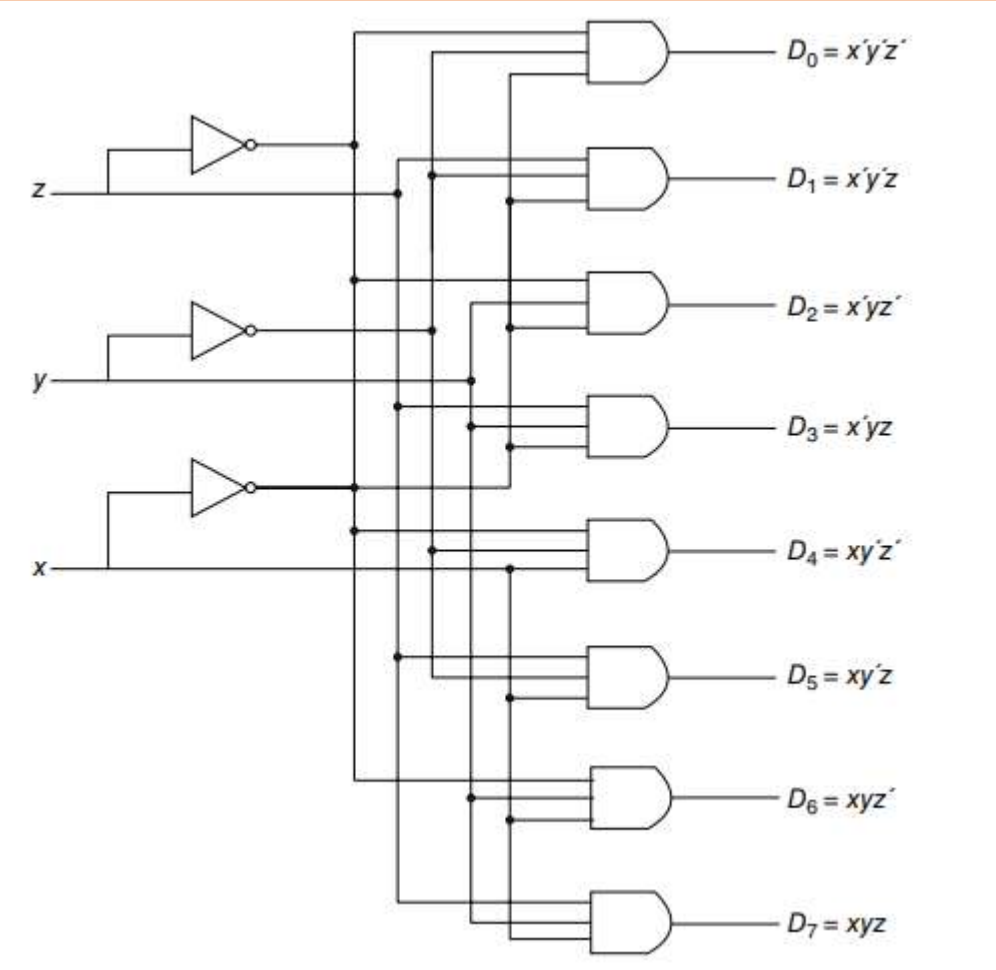


❖ **Decoders:** for 'n' bit input we have 2^n output.

- **3-to-8-line decoder.**
  - ✓ **Truth table:**

Inputs			Outputs							
x	y	z	D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

✓ **Circuit diagram:**



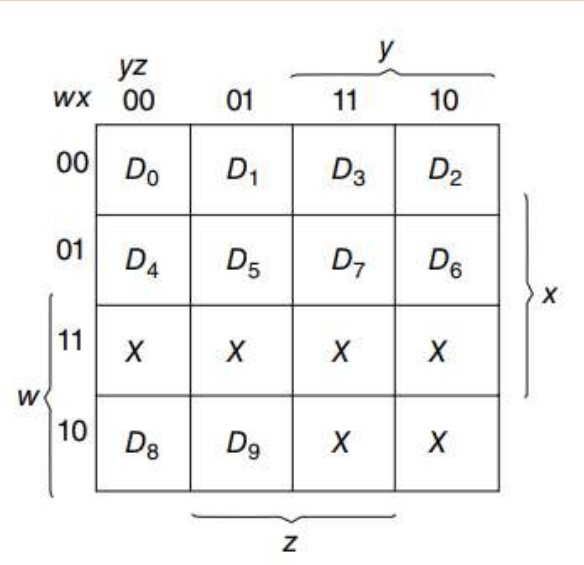
11th lecture(27/02/2023)

❖ **BCD to Decimal decoder**

➢ **Truth table:**

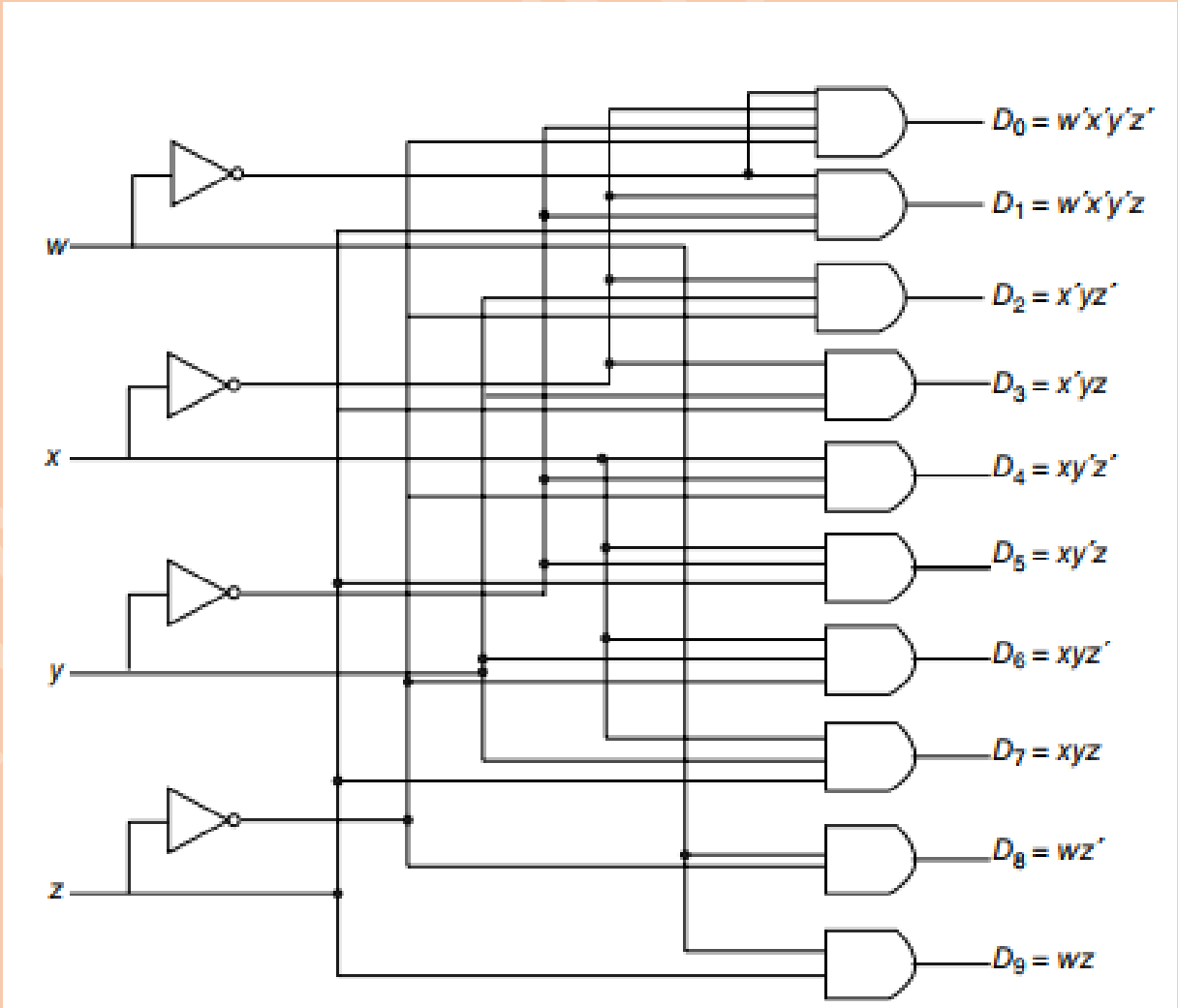
W	X	Y	Z	D0	D1	D2	D3	D4	D5	D6	D7	D8	D9
0	0	0	0	1	0	0	0	0	0	0	0	0	0
0	0	0	1	0	1	0	0	0	0	0	0	0	0
0	0	1	0	0	0	1	0	0	0	0	0	0	0
0	0	1	1	0	0	0	1	0	0	0	0	0	0
0	1	0	0	0	0	0	0	1	0	0	0	0	0
0	1	0	1	0	0	0	0	0	1	0	0	0	0
0	1	1	1	0	0	0	0	0	0	1	0	0	0
1	0	0	0	0	0	0	0	0	0	0	1	0	0
1	0	0	1	0	0	0	0	0	0	0	0	1	0
1	0	1	0	0	0	0	0	0	0	0	0	0	1

- **Simplify this using k-map:**



$D_0 = w'x'y'z'; D_1 = w'x'y'z; D_2 = x'yz'; D_3 = x'yz; D_4 = xy'z'; D_5 = xy'z; D_6 = xyz'; D_7 = xyz; D_8 = wz'; D_9 = wz$

- **Logic diagram:**

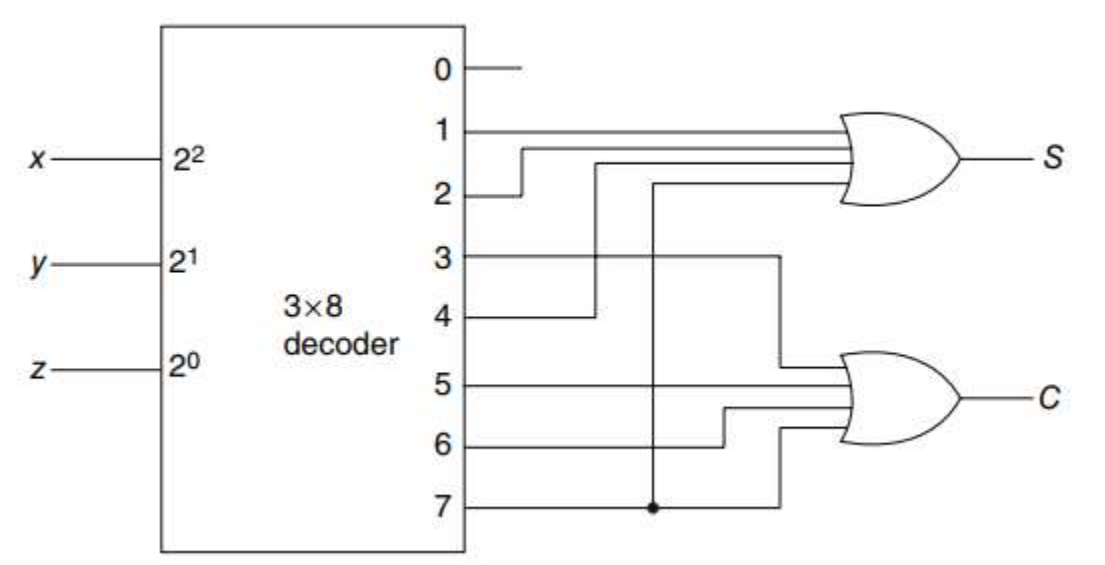


❖ **3x8 decoder.**

From full adder we find:

Sum = x' y' z+x' yz+xy' z'+xyz

Carry=x'yz+xy'z+xyz'+xyz



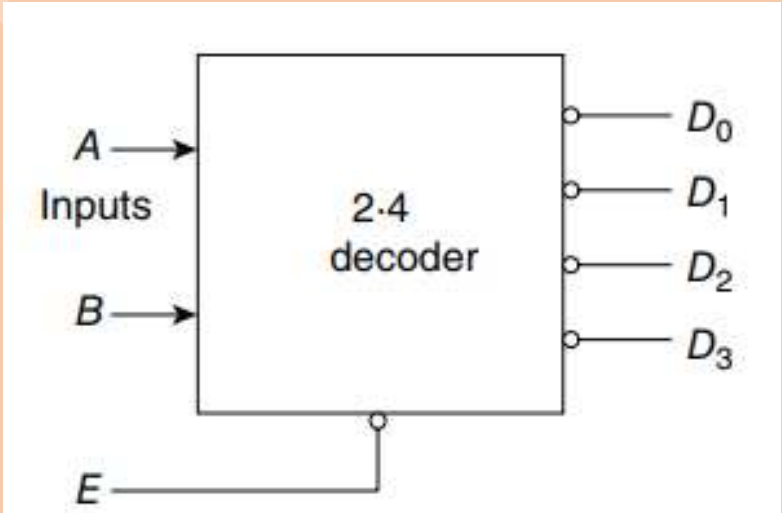
❖ **2x4 line decoder:**

➢ **Truth table:**

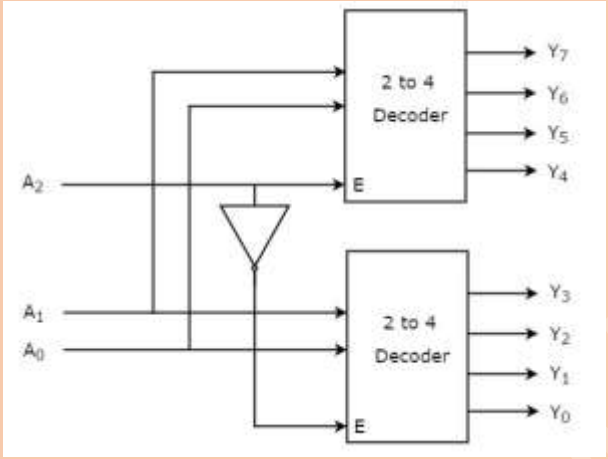
X	Y	D0	D1	D2	D3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

D0=x'y';D1=x'y;D2=xy';D3=xy

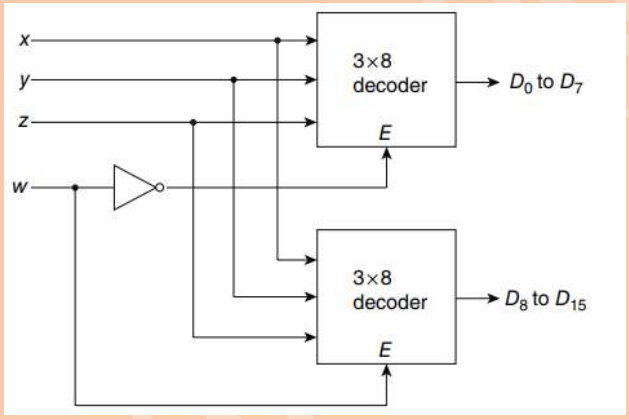
➢ **Block diagrams:**



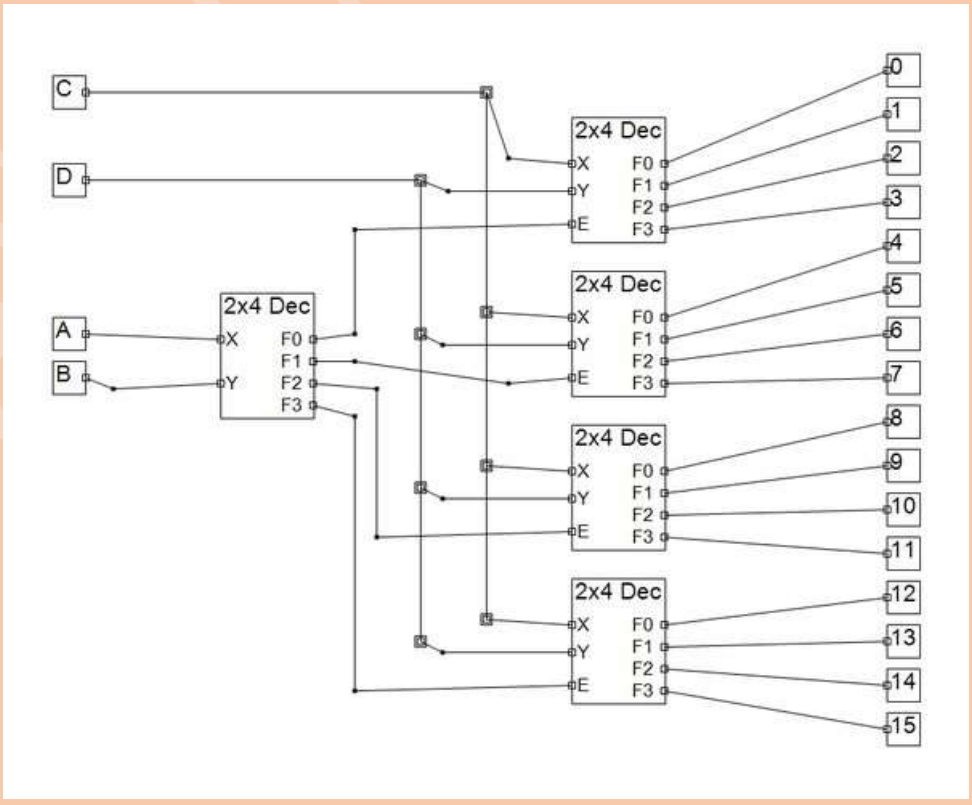
❖ 3 to 8 Decoder using 2 to 4 decoders:



❖ A 4 x 16 decoder constructed with two 3 x 8 decoders:



❖ A 4 x 16 decoder constructed with five 2x4 decoders:



12th lecture(05/03/2023)

❖ Demultiplexers: Decoder with an enable input.

- **Decoder** is a combinational circuit that has 'n' input lines and maximum of 2^n output lines.

- **2 to 4 decoder:** Let 2 to 4 Decoder has two inputs A_1 & A_0 and four outputs Y_3, Y_2, Y_1 & Y_0.
  One of these four outputs will be '1' for each combination of inputs when enable, E is '1'.
  - The **Truth table** of 2 to 4 decoder is shown below.

Enable	Inputs		Outputs			
E	A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	x	x	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

From Truth table, we can write the **Boolean functions** for each output as

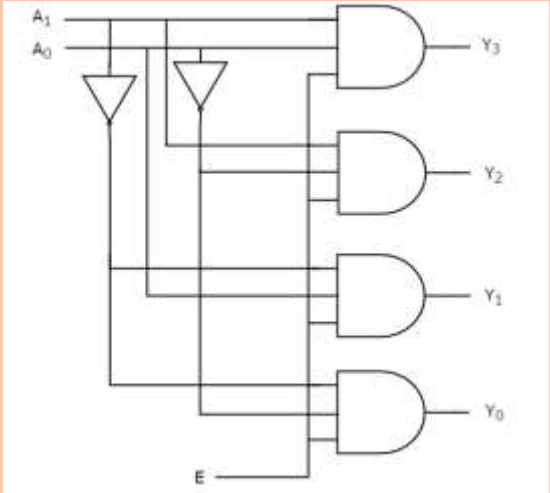
$Y3=E.A1.A0$

$Y2=E.A1.A0'$

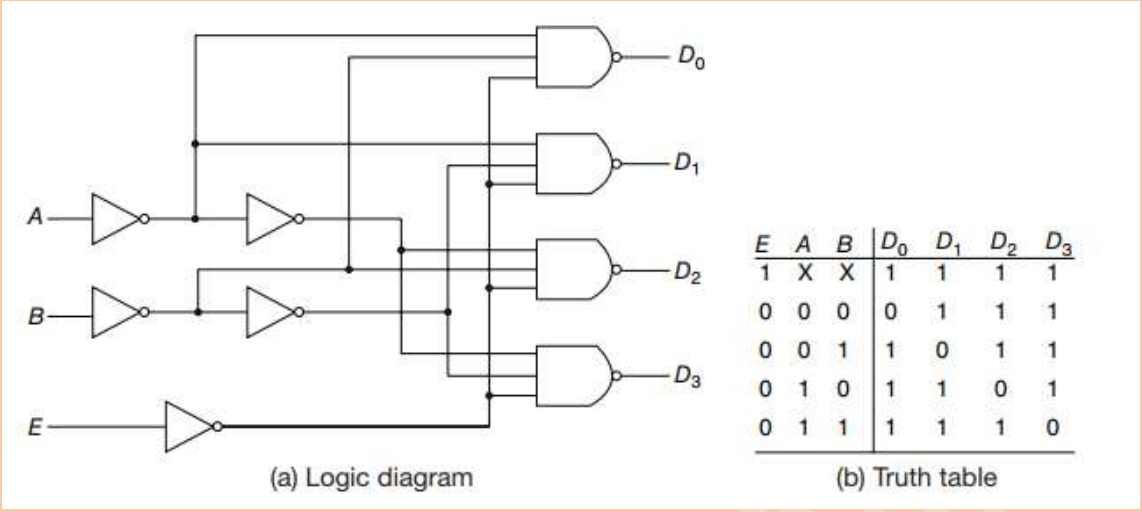
$Y1=E.A1'.A0$

$Y0=E.A1'.A0'$

➢ Logic diagram:



➢ **With NAND gate:**



(a) Logic diagram

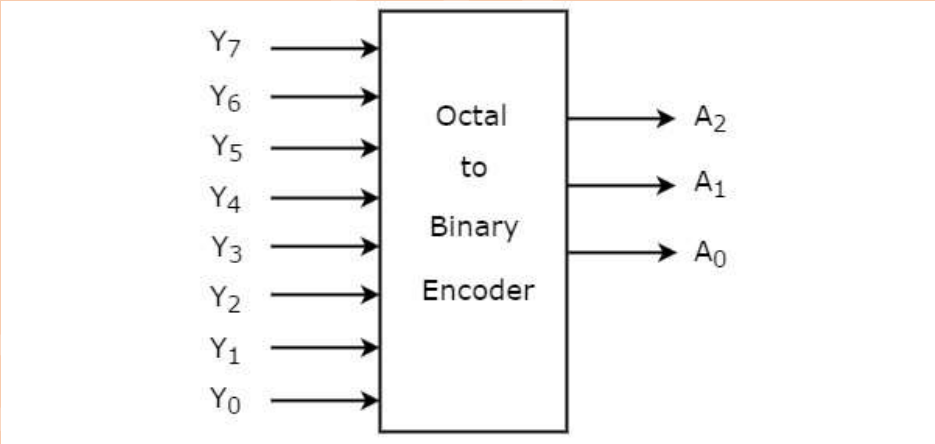
E	A	B	D_0	D_1	D_2	D_3
1	X	X	1	1	1	1
0	0	0	0	1	1	1
0	0	1	1	0	1	1
0	1	0	1	1	0	1
0	1	1	1	1	1	0

(b) Truth table

❖ **Encoder:** An Encoder is a combinational circuit that performs the reverse operation of Decoder. It has maximum of 2^n input lines and 'n' output lines.

❖ **Octal to binary encoder:** Octal to binary Encoder has eight inputs, Y_7 to Y_0 and three outputs A_2, A_1 & A_0. Octal to binary encoder is nothing but 8 to 3 encoder.

➢ The **block diagram** of octal to binary Encoder is shown in the following figure:



➢ The **Truth table** of octal to binary encoder is shown below:

Inputs								Outputs		
Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0	A_2	A_1	A_0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0

0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

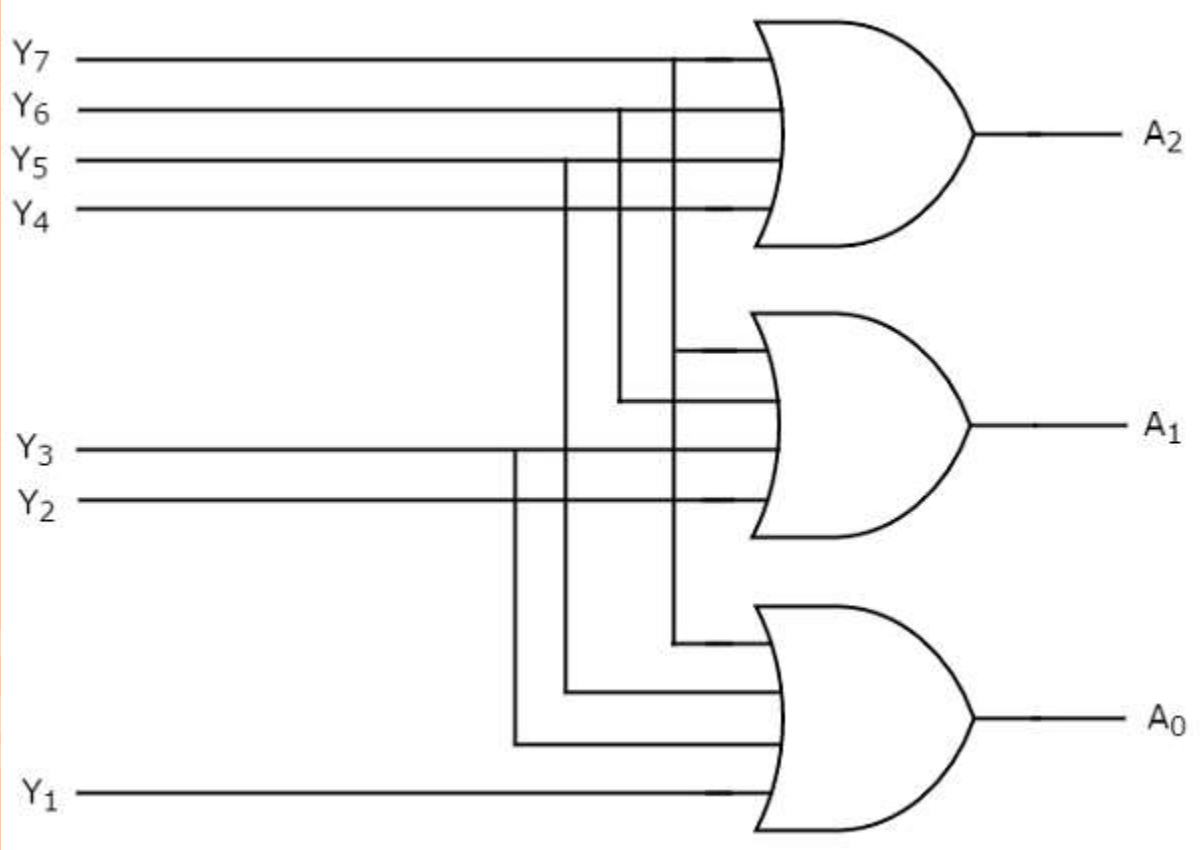
From Truth table, we can write the **Boolean functions** for each output as:

A2=Y7+Y6+Y5+Y4

A1=Y7+Y6+Y3+Y2

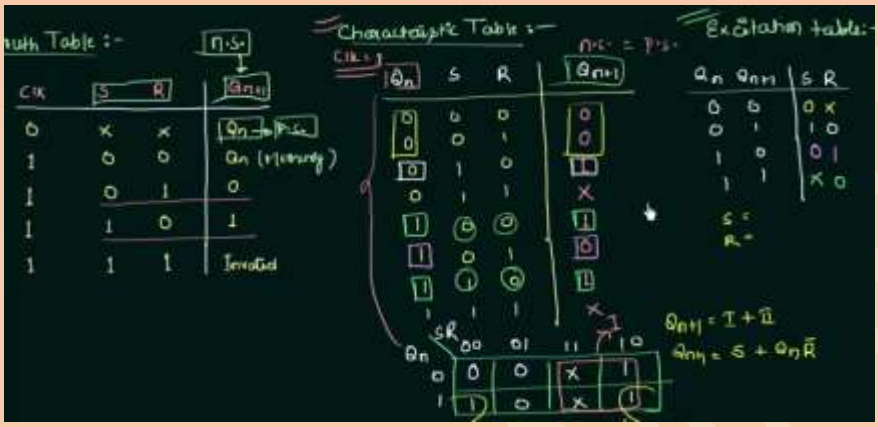
A0=Y7+Y5+Y3+Y1

- **The circuit diagram of octal to binary encoder is shown in the following figure.**

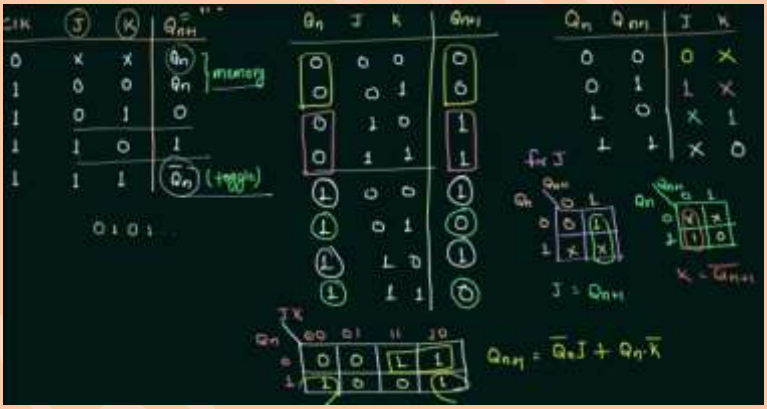


13th lecture (10/05/2023)-from slide, not important may be.

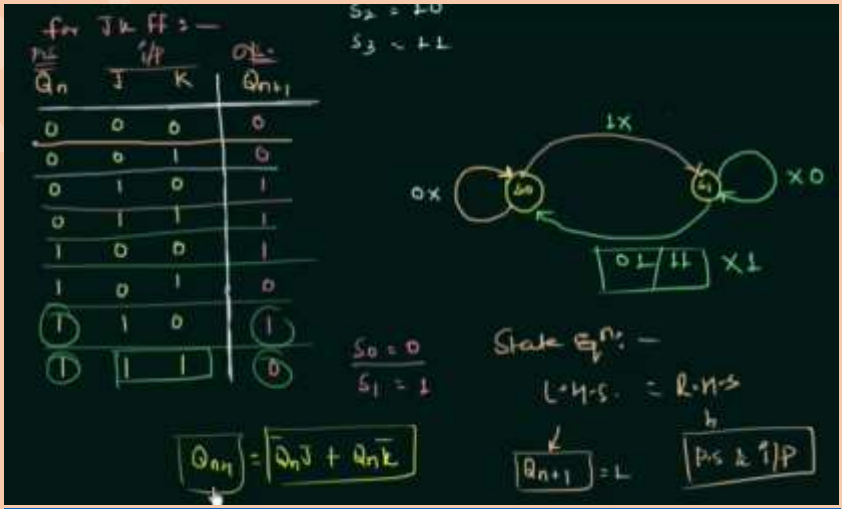
- **This lecture is probably not important for exam hence I noted it another file.**
- **SR flip-flop.**
  - ✓ https://www.youtube.com/watch?v=uiKKRPZbuXA



- **JK flip-flop.**
  - ✓ https://www.youtube.com/watch?v=lnQD2_M9uDI&t=251s

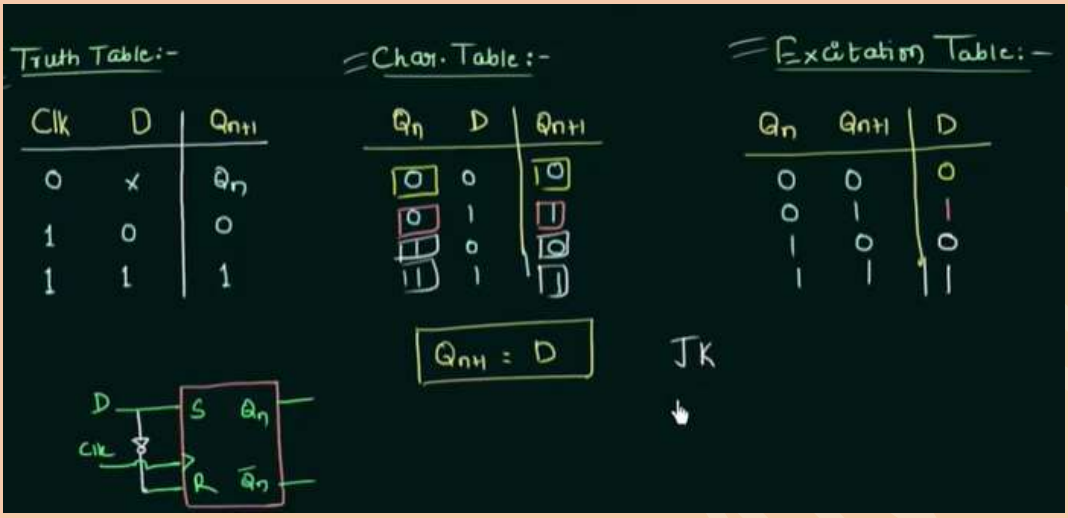


- **State diagram and state table.**
  - ✓ https://www.youtube.com/watch?v=-F2gXZVJuek

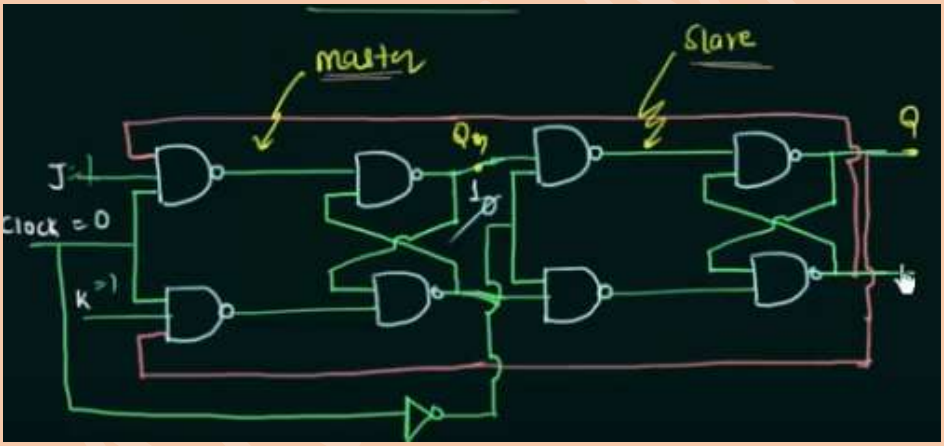


❖ **D flip-flop.**

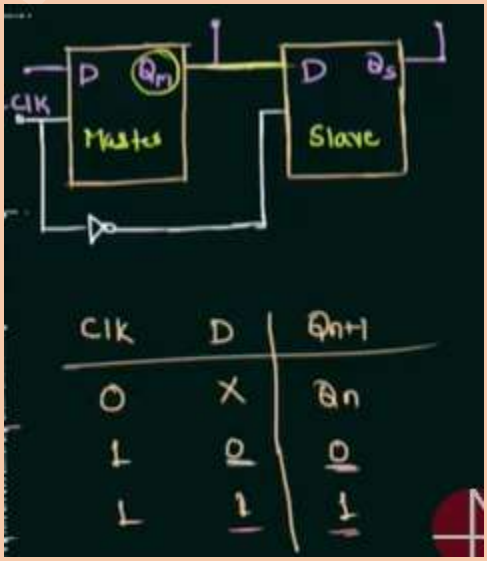
✓ https://www.youtube.com/watch?v=4c6z9RKrC8Q



❖ **Master Slave JK Flip Flop:**

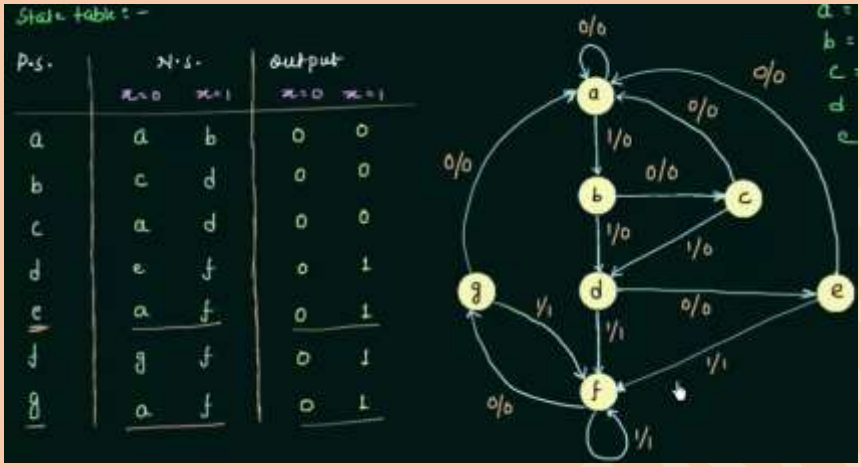


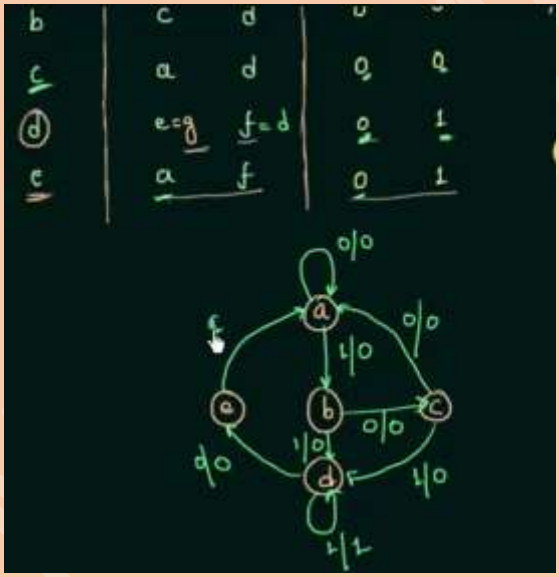
❖ **Master Slave D Flip Flop:**



❖ **State Reduction and Assignment:**

✓ https://www.youtube.com/watch?v=IXORIs1dHs0&t=1s

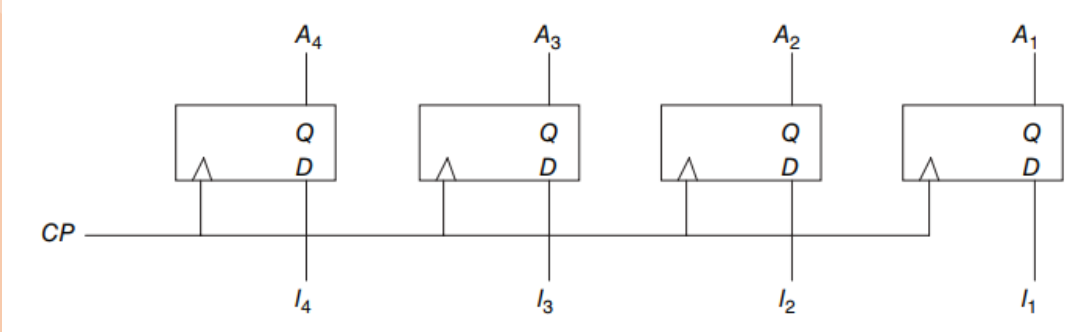




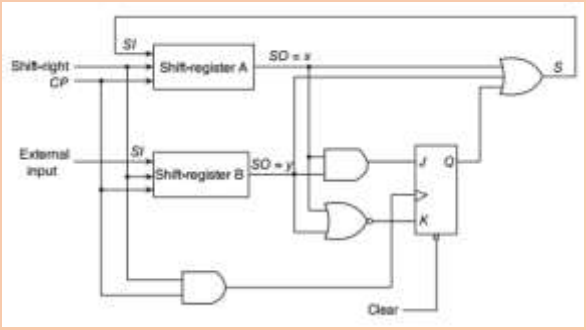
14th lecture (15/05/2023)

❖ **Register:** A register is a group of binary storage cells suitable for holding binary information. A group of flip-flops constitutes a register, since each flip-flop is a binary cell capable of storing one bit of information.

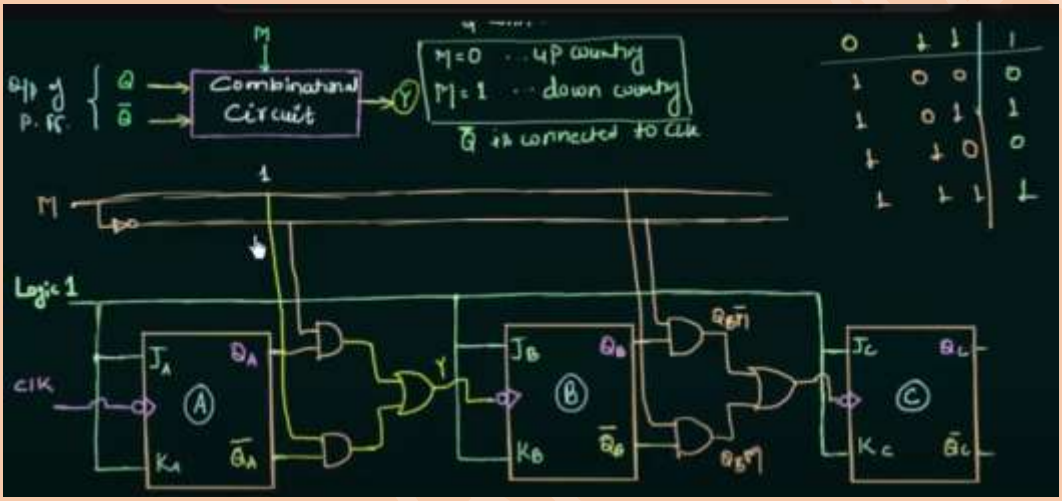
❖ **4-bit register:**



❖ **Ripple counter******(1 question from here):**



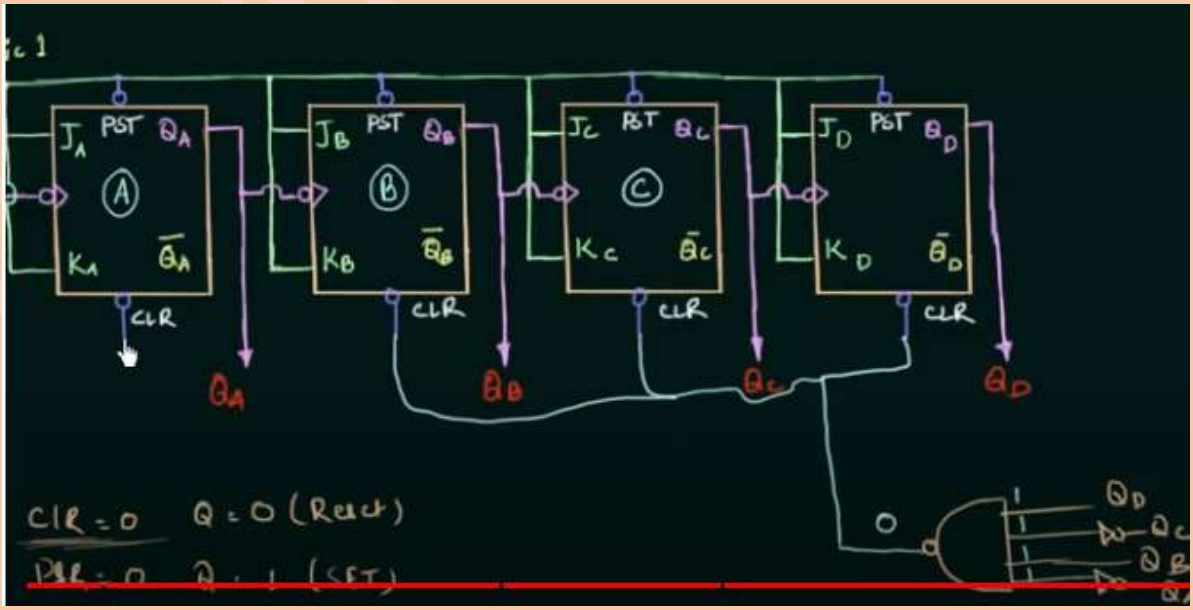
➢ **3/4-bit binary ripple counter:**



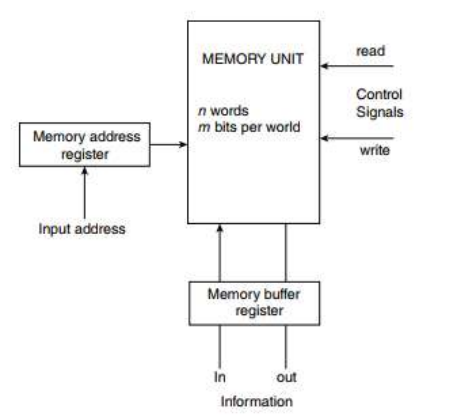
✓ https://www.youtube.com/watch?v=5Um3NDvsYjQ&t=93s

➢ **BCD Ripple counter:**

- https://www.youtube.com/watch?v=fKVZpupyP_o

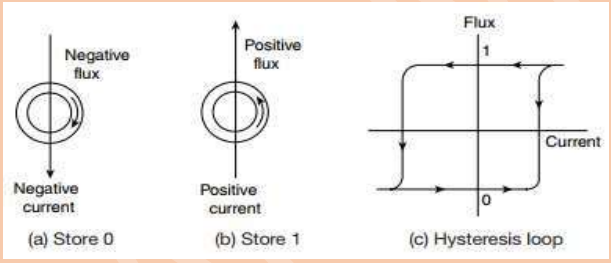


❖ **Memory unit:**



❖ **IC memory:** Memory IC means any Integrated Circuit that is configured to store bits of data in memory cells within a memory array and that has as its primary purpose the storage and retrieval of such electronic data.

❖ **Magnetic core memory:**



❖ **RTL:** it is used to describe the micro-operations transfer among registers.

❖ **Inter-register Transfer:** It can send data and instructions from one register to another register, memory to register, and memory to memory, the register transfer approach is used.